

Haneen Alaa Hassan

Depi Day7 C#

1. Why does defining a custom constructor suppress the default constructor in C#?

When a class does not define any constructor, C# automatically provides a parameterless default constructor.

However, when we define our own constructor, C# assumes that we want to control how objects are initialized. Therefore, the compiler does not automatically generate the default constructor.

If we still need a parameterless constructor, we must define it explicitly.

2. How does method overloading improve code readability and reusability?

Method overloading allows us to use the same method name for different parameter lists.

This improves readability because related operations have the same meaningful name instead of using different names.

It also improves reusability because the same method can work with different numbers or types of parameters.

For example, we can have:

Sum(int, int)

Sum(int, int, int)

Sum(double, double)

All of them perform addition but accept different parameters.

3. What is the purpose of constructor chaining in inheritance?

Constructor chaining allows a derived class constructor to call the constructor of its base class using the base keyword.

It ensures that the inherited properties are initialized correctly before initializing the properties of the derived class.

It also avoids duplicating initialization code.

Example:

```
public Child(int x, int y, int z) : base(x, y)
{
    Z = z;
}
```

Here, the Child constructor calls the Parent constructor first.

4. How does new differ from override in method overriding?

The new keyword is used to hide a method from the base class.

The override keyword is used to replace the implementation of a virtual or abstract method in the derived class.

With new, the method that is called depends on the reference type.

With override, the method that is called depends on the actual object type at runtime.

Therefore, override supports runtime polymorphism, while new only hides the base method.

5. Why is ToString() often overridden in custom classes?

ToString() is overridden to provide a meaningful string representation of an object.

The default implementation usually returns the class name, which is not very useful when displaying object information.

By overriding ToString(), we can display important properties of the object in a readable format.

For example:

```
public override string ToString()
{
    return $"({X}, {Y})";
}
```

6. Why can't you create an instance of an interface directly?

An interface cannot be instantiated directly because it only defines a contract and does not represent a complete object implementation.

A class must implement the interface before an object can be created.

For example:

```
IShape shape = new Rectangle(10, 5);
```

Here, we cannot write:

```
IShape shape = new IShape();
```

because IShape is an interface, not a concrete class.

7. What are the benefits of default implementations in interfaces introduced in C# 8.0?

Default interface implementations allow an interface to contain a method with an implementation.

They provide several benefits:

They allow adding new methods to an existing interface without immediately breaking all implementing classes.

They allow interfaces to provide common behavior.

They support backward compatibility.

They reduce duplicated code in implementing classes.

For example:

```
void PrintDetails()  
{  
    Console.WriteLine("This is a shape.");  
}
```

A class implementing the interface can use this method without providing its own implementation.

8. Why is it useful to use an interface reference to access implementing class methods?

Using an interface reference allows us to work with objects based on their common contract rather than their specific class.

For example:

```
IMovable car = new MovableCar();  
car.Move();
```

This provides abstraction and flexibility.

It also supports polymorphism because different classes can implement the same interface and can be handled using the same interface reference.

9. How does C# overcome the limitation of single inheritance with interfaces?

The class can implement both Read() and Write().

Therefore, interfaces allow a class to have multiple contracts or behaviors without using multiple class inheritance.

10. What is the difference between a virtual method and an abstract method in C#?

A virtual method has a default implementation in the base class, and the derived class can optionally override it.

An abstract method C# allows a class to inherit from only one base class, but a class can implement multiple interfaces.

For example:

```
class File : IReadable, IWritable
```

does not have an implementation in the base class, and the derived class must implement it.

Example of a virtual method:

```
public virtual void Draw()  
{  
    Console.WriteLine("Drawing Shape");  
}
```

Example of an abstract method:

```
public abstract double CalculateArea();
```

Also, a class containing an abstract method must be declared as abstract.

11. What is the difference between a class and a struct in C#?

A class is a reference type, while a struct is a value type.

When we create an object of a class, the variable normally contains a reference to the object.

A struct contains its value directly.

Example:

```
class Person  
{  
    public string Name;  
}
```

and:

```
struct Point  
{  
    public int X;  
    public int Y;  
}
```

A struct is generally suitable for small values that represent a single piece of data, such as a point, coordinate, or measurement.

12. If inheritance is a relation between classes, clarify other relations between classes

There are several important relationships between classes in Object-Oriented Programming:

1. Inheritance – IS-A

Inheritance means that one class derives from another class.

Example:

Car IS-A Vehicle

The derived class inherits members from the base class.

2. Association – USES / KNOWS

Association means that one class is related to or uses another class.

Example:

Doctor ---- Patient

A doctor can interact with or treat a patient.

3. Aggregation – HAS-A

Aggregation represents a weak HAS-A relationship.

The contained object can exist independently of the container.

Example:

Department HAS-A Teacher

If the department is removed, the teacher can still exist independently.

4. Composition – HAS-A

Composition is a strong HAS-A relationship.

The contained object is strongly associated with the lifetime of the containing object.

Example:

House HAS-A Room

The room is considered a part of the house.

Quick way to remember the relationships

Inheritance → IS-A

Association → USES / KNOWS

Aggregation → HAS-A (weak relationship)

Composition → HAS-A (strong relationship)